TECHNICAL SPECIFICATION · V0.1

WeatherHub Backend Architecture

Multi-tenant NestJS API and MQTT ingest worker serving real-time environmental data from ESP32 weather stations across university campuses. Each tenant is physically isolated in its own Postgres database; devices, users, and data never cross tenant boundaries.

NestJS 10

TypeScript 5

TypeORM 0.3

Postgres 16 · TimescaleDB

Eclipse Mosquitto 2

Espressif ESP32

JWT (HS256)

Docker Compose

Hedera (Phase 2)

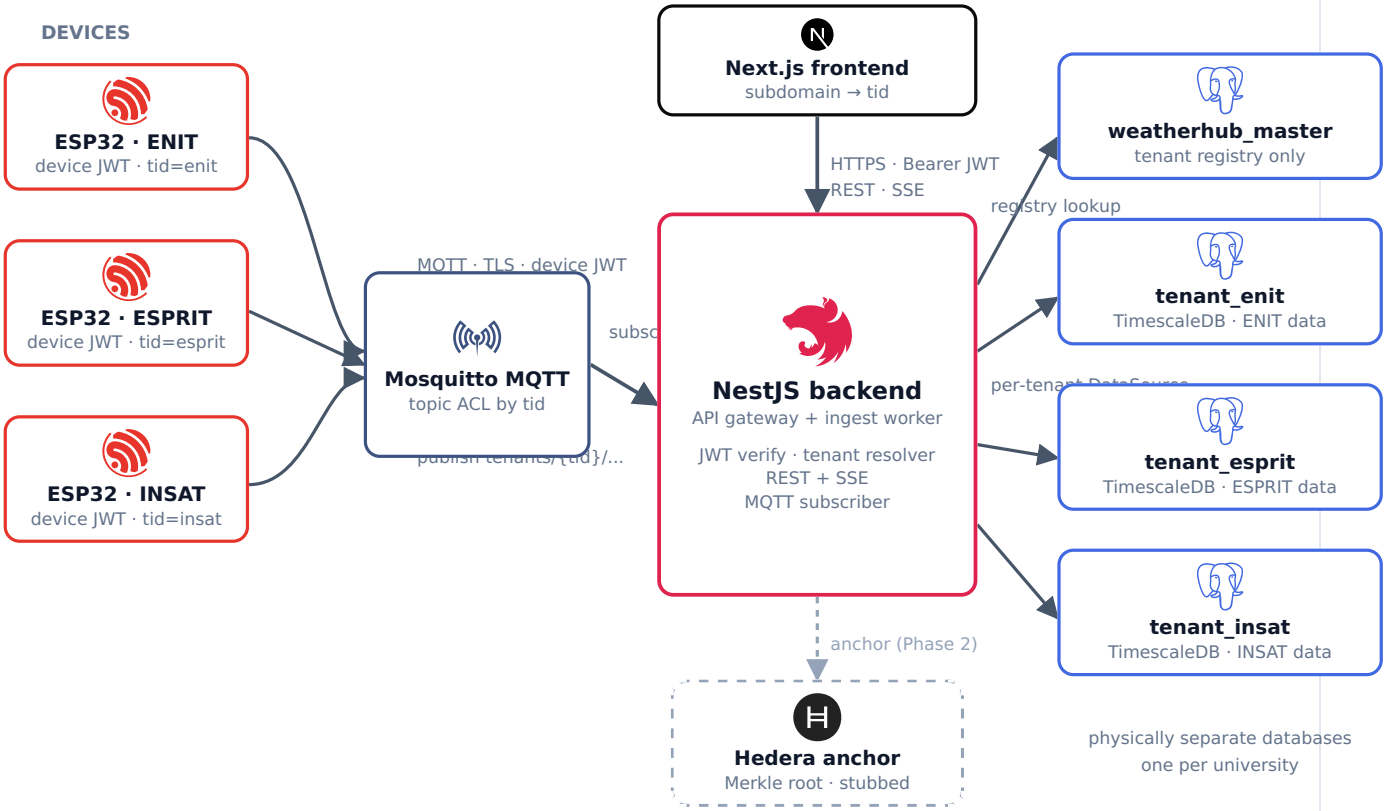
Next.js 16 frontend

Service: `weatherhub-backend` · Default port `:3001`
Tenancy: database-per-tenant · Ingest: MQTT v3.1.1 (TLS-ready)
Tenant resolution: subdomain · `X-Tenant` header · JWT `tid` claim

SECTION 1

System overview

The diagram below shows how a single backend deployment serves multiple independent universities. Each university is a **tenant**; its devices publish to dedicated MQTT topics, its users authenticate against its own user table, and its sensor readings live in a database that no other tenant can reach.



All inter-service traffic is TLS-terminated at the edge in production.

SECTION 2

Tenancy model

WeatherHub uses **database-per-tenant (DPT)**. Each university gets a physical Postgres database named `tenant_<slug>`. A separate **master database** (`weatherhub_master`) holds only the tenant registry — never sensor data, never users.

Why DPT and not row-level `tenant_id`?

- **Data sovereignty.** Universities frequently require that their data physically lives in a database they can audit, back up, or extract in isolation. Row-level isolation cannot offer the same guarantee.
- **Blast radius.** A bad query, leaked credential, or RLS policy mistake in a row-level model can leak every tenant. In DPT, those failures stay contained to one database.
- **Lifecycle simplicity.** "Delete this tenant" is a single `DROP DATABASE`. Backup, restore, and migration to a separate Postgres host are also trivial.

What lives where

DATABASE	TABLES	PURPOSE
`weatherhub_master`	`tenants`	Registry: slug, name, dbName, optional emailDomain, Hedera account, active flag.
`tenant_enit` (one per university)	`users` `stations` `readings` (hypertable) + future: alerts, forecasts, integrity_batches	Everything tenant-specific. `readings` is a TimescaleDB hypertable partitioned by `recordedAt` for efficient time-series queries and continuous aggregates.

Tenant resolution

Three sources, evaluated in this order:

1. **JWT `tid` claim.** Authoritative for any authenticated request. Cannot be spoofed without the signing secret.
2. **`X-Tenant` header.** Used for tools, the login endpoint, and tests. Set explicitly by the caller.
3. **Host subdomain.** `enit.weatherhub.local` → tenant `enit`. Convenient for browser sessions and per-tenant branding.

A `TenantContextMiddleware` populates `req.tenant` from sources 2 and 3 before any guard runs. The `JwtStrategy` overrides it from the `tid` claim on every authenticated route so an attacker cannot mix a valid JWT for tenant A with a header for tenant B.

Tenant DataSources are pooled, lazy, and cached

`TenantService.getDataSource(slug)` returns a singleton TypeORM `DataSource` per tenant. The first call to a given slug looks up the connection string in master, opens the pool, and caches the `DataSource` in memory. Subsequent calls reuse it. The service closes every pool on application shutdown.

> **Operational consequence.** Backend memory grows linearly with the number of active tenants (each pool defaults to 10 connections). At thousands of tenants, switch to a connection-string multiplexer (pgbouncer / RDS Proxy) rather than holding pools in-process.

SECTION 3

Authentication & request flow

Login

1. **Resolve tenant.** If `req.tenant` is set by middleware, use it. Otherwise look up the tenant by the email's domain (`chiheb@enit.utm.tn` → tenant `enit`). If neither succeeds, return `400` .
2. **Open the tenant's DataSource** and read the user by email.
3. **Bcrypt-compare** the supplied password against `passwordHash` . Wrong password and unknown user return the same generic `401` message — no user enumeration.
4. **Update** `lastLoginAt` .
5. **Issue a JWT** with payload `{ sub, tid, email }` , signed HS256 with `JWT_SECRET` . Default expiry 24 h.

JWT payload

```
{
  "sub":   "<user uuid>",
  "tid":   "<tenant slug>",       // authoritative on every authenticated request
  "email": "chiheb@enit.utm.tn",
  "iat":   1778776080,
  "exp":   1778862480
}
```

Every authenticated request

1. **JWT extracted** from `Authorization: Bearer …` header. The `JwtStrategy` also accepts `?token=<jwt>` as a fallback so browser `EventSource` (SSE) can authenticate without setting headers.
2. **Signature verified.** If `tid` or `sub` is missing or malformed → `401` .
3. **Tenant re-resolved** from the `tid` claim. Inactive tenants → `401` .
4. **User reloaded from the tenant DB**. Immediate revocation works: deleting or disabling the user invalidates the token on the next request.
5. `req.user` **and** `req.tenant` are populated for handlers and decorators.

Roles

Each user has a role: `admin` , `researcher` , or `viewer` . The role lives in the tenant DB so it can be revoked or upgraded without re-issuing JWTs. Route-level authorization will be added in the alerts phase via a `@Roles(...)` decorator on top of `JwtAuthGuard` .

SECTION 4

Ingest path (ESP32 → tenant DB → SSE)

1. Device → MQTT

Each ESP32 holds a long-lived **device JWT** (separate signing secret from user JWTs) carrying `{ deviceId, stationId, tid }`. The device authenticates to Mosquitto by placing the JWT in the MQTT username and password fields. The broker's ACL enforces that the device may only publish to topics whose `tid` segment matches the token's `tid` claim.

```
topic:   tenants/{tid}/stations/{sid}/readings
payload: { "recordedAt": "...", "temperatureC": 24.5, "humidityPct": 55, ... }
```

2. MQTT → NestJS ingest worker

A NestJS worker subscribes with the wildcard pattern `tenants/+/stations/+/readings`. For every message it:

1. Re-verifies the device JWT signature.
2. Validates the payload against the `IngestReadingDto` schema (same validation used by the manual `POST /readings` path).
3. Resolves the tenant DataSource for `tid`.
4. Inserts into the tenant's `readings` hypertable.
5. Updates the station's `lastSyncedAt` and `status` via `StationsService.touchLastSync()`.
6. Publishes to the in-memory SSE channel keyed by `(tenantSlug, stationId)`.

> **Phase status.** The MQTT subscriber itself is not yet implemented. The data path above is exercised today via the `POST /readings` REST endpoint (admin-authenticated), which calls the same `ReadingsService.ingest()` method the MQTT worker will use.

3. SSE fan-out

`ReadingsStreamService` maintains a `Map<tenantSlug:stationId, Subject>`. When a reading is persisted, all browser clients subscribed to that station receive it on their open SSE connection within a single round-trip. The frontend's `useSSEStream` hook reconnects automatically on failure.

> **Scaling note.** The subscriber registry is in-process. Two backend replicas would each see only the clients connected to them. Once horizontal scaling is needed, replace the in-memory map with a Redis or NATS pub/sub adapter behind the same `ReadingsStreamService` interface.

SECTION 5

Data model

Master DB · `tenants`

COLUMN	TYPE	NOTES
`id`	uuid PK	
`slug`	varchar(64), unique	URL-safe identifier (`enit` , `esprit` ...).
`name`	varchar(200)	Display name.
`location`	varchar(200), nullable	
`emailDomain`	varchar(128), unique partial	Enables tenant resolution by email.
`dbName`	varchar(128)	Physical Postgres database name.
`hederaAccountId`	varchar(64), nullable	Reserved for Phase 2 anchoring.
`active`	boolean	Soft-deactivation flag.

Tenant DB · core tables

TABLE	SHAPE	NOTES
`users`	uuid PK · email (unique) · passwordHash · role · lastLoginAt	Lives inside the tenant DB — no shared user pool.
`stations`	uuid PK · name · location · lat/long · status · lastSyncedAt · enabledSensors[]	`enabledSensors` is a Postgres text[]; values come from a closed set in the entity.
`readings`	composite PK (id, recordedAt) · stationId · per-metric columns · receivedAt	TimescaleDB hypertable. Aggregations use `time_bucket()` for 5m/15m/1h/1d intervals.

Migrations

Two migration sets are tracked separately: master migrations in `migrations/master/` apply once; tenant migrations in `migrations/tenant/` must be applied to every tenant database. The `tenant:migrate` CLI iterates the registry and runs pending migrations on each active tenant. The `tenant:provision` CLI creates a new database, installs `timescaledb` and `uuid-ossp` extensions, applies tenant migrations, and seeds an admin user in one shot.

Operational checklist before exposing to the internet

- **JWT secrets.** Replace `.env` placeholders with `openssl rand -base64 48` ; rotate via a key-ID claim if you need overlap.

- **Mosquitto auth.** Switch `allow_anonymous false`, ship a `passwd` file and ACL list mapping device tokens to per-tenant topic patterns. Terminate TLS on `:8883`.
- **Postgres roles.** The application user only needs `CONNECT` + table CRUD on its tenant DB; `CREATE DATABASE` should be reserved for a separate provisioning role.
- **Backups.** One `pg_dump` per tenant DB; the registry must be backed up before tenant DBs so a restore preserves referential integrity.
- **Observability.** Add the `tid` as a structured log field on every request — without it, multi-tenant logs are unreadable.
- **Rate limiting.** Apply per-tenant and per-device limits at the edge (broker + API gateway), not in the NestJS process.

What works today. Tenant provisioning, login & `/auth/me`, stations + readings REST, `time_bucket()` aggregation, manual `POST /readings` ingestion, and SSE live streaming. The frontend can run against this backend by flipping `NEXT_PUBLIC_USE_MOCK_DATA=false`.

Not yet wired. MQTT subscriber, alerts evaluator + ack/resolve, forecast scheduler, Hedera anchoring. Each is an additive module: no changes to tenancy, auth, or storage are required to land them.